计算机科学与技术学院　大数据管理与分析　课程实验报告

实验题目：熟悉常用的 Linux、Hadoop 与 HDFS 操作及 MapReduce 初级编程		学号：202600000000
日期：2026.9.22	班级：2026 级 1 班	姓名：张三
Email：XXXXXXXX@qq.com		

实验目的：

（1）熟悉 Linux 常用的文件与目录操作命令以及文本查看命令，能够脱离图形界面完成日常的文件管理，为在 Linux 上运行和调试 Hadoop 打下基础。

（2）掌握 Hadoop 伪分布式环境的启动与验证方法，熟悉 HDFS 的基本 Shell 操作（创建目录、上传、下载、查看文件列表），理解 Linux 本地文件系统与 HDFS 之间的数据搬运过程。

（3）理解 HDFS 在 Hadoop 体系结构中的角色，能够使用 HDFS Java API 实现上传（追加/覆盖）、下载（自动重命名）、查看内容、查看属性、递归列目录、创建与删除文件/目录、首尾追加、移动等 10 项操作，并给出每一项与之等价的 Shell 命令。

（4）掌握 MapReduce 的编程模型与作业运行流程，能够用 MapReduce 解决数据去重、全局排序和表连接（祖孙关系挖掘）三类典型问题，理解 shuffle 阶段“按 key 排序、按 key 分组”的作用。

实验软件和硬件环境：

硬件：x86_64 虚拟机（VMware Workstation 17），分配 4 核 CPU、4 GB 内存、40 GB 磁盘，宿主操作系统为 Windows 11。

软件：Ubuntu 20.04.6 LTS（64 位）；OpenJDK 1.8.0_362；Hadoop 3.3.4，伪分布式部署，`core-site.xml` 中 `fs.defaultFS = hdfs://localhost:9000`，`hdfs-site.xml` 中 `dfs.replication = 1`；Maven 3.8.6（依赖 `hadoop-client` 3.3.4，scope 为 provided）；文本编辑器 Vim / Gedit；浏览器用于查看 NameNode Web UI（`http://localhost:9870`）。

实验原理和方法：

（1）HDFS 的体系结构。HDFS 采用主从结构：NameNode 负责维护文件系统的命名空间（目录树、文件与数据块的映射）以及每个数据块所在的 DataNode 列表；DataNode 负责存储实际的数据块，默认块大小 128 MB、默认 3 副本（本实验伪分布式下副本数为 1）。HDFS 面向“一次写入、多次读取”的大文件场景，不支持随机修改，只能在末尾追加或整体重写。客户端通过 `FileSystem` 抽象类访问文件

系统：FileSystem.get(new URI("hdfs://localhost:9000"), conf, "hadoop") 返回的实际上是 DistributedFileSystem 对象。

（2）HDFS Shell 与 Java API 的关系。hdfs dfs 子命令本质上是 FileSystem API 的封装：-put 对应 copyFromLocalFile，-get 对应 copyToLocalFile，-cat 对应 open 加流拷贝，-ls 对应 listStatus，-mkdir -p 对应 mkdirs，-rmdir 对应 recursive=false 的 delete，-appendToFile 对应 append，-mv 对应 rename。因此本实验对每一项功能都给出 Java 实现与 Shell 实现两套方案，互相印证。

（3）MapReduce 计算模型。一个作业由 Mapper、Reducer 和 Driver（main 方法）组成。输入文件被切分为若干 InputSplit，每个分片交给一个 map 任务；map 输出的 (k2, v2) 先写入环形缓冲区，溢写时按 key 排序并分区，再合并成有序文件；reduce 端通过 shuffle 拉取属于自己的分区，对相同 key 的记录做归并排序与分组，把同一 key 的 value 组织成迭代器交给 reduce 处理一次，结果写入 HDFS。框架“按 key 排序 + 按 key 分组”正是三个程序成立的关键：去重靠“相同 key 只 reduce 一次”，排序靠“key 天然有序”，连接靠“相同 join key 落到同一个 reduce”。

（4）三个问题的建模。① 去重：把整行内容作为 key、value 置为 NullWritable，reduce 只输出 key，重复行自然只剩一条；合并多个文件只需让输入路径指向多个文件。需要注意样例文件 A、B 中字段间的空格个数不同，若直接用原始行做 key，20170101　x 与 20170101　x 会被当成两条不同记录，因此 map 阶段先用 replaceAll("\\s+", " ") 做空白归一化。② 排序：把整数本身作为 key，借用 shuffle 的排序能力；名次是全局概念，必须让所有 key 进入同一个 Reducer，用成员变量累加计数。③ 祖孙关系：把 child-parent 看作二元关系 R(child, parent)，祖孙关系就是 R 与自身的连接，连接条件为 R1.parent = R2.child；map 阶段对每条记录发出两条带标记的记录，reduce 阶段对同一 key 下的“孙辈候选”与“祖辈候选”做笛卡尔积。

实验步骤：（不要求罗列完整源代码）

一、常用 Linux 命令

按实验内容逐条执行（脚本 scripts/01_linux_ops.sh），关键命令与结果说明如下：

1. cd：cd /usr/local 切换到指定目录；cd .. 回到上一级（/usr）；cd ~ 回到当前用户主目录 /home/hadoop，三者分别是绝对路径、相对路径和家目录简写。

2. ls：ls -l /usr 以长格式列出 /usr 下全部文件与目录，可见 bin、lib、

local、share 等。

3. mkdir：cd /tmp && mkdir a 建立目录 a；mkdir -p a1/a2/a3/a4 用 -p 一次建立多级目录，不加 -p 会因父目录不存在而报错。

4. rmdir：rmdir /tmp/a 删除空目录 a；rmdir -p a1/a2/a3/a4 由内向外逐级删除，也可从最内层开始逐个 rmdir。

5. cp：cp ~/.bashrc /usr/bashrc1 复制文件并重命名（写 /usr 需 sudo）；mkdir /tmp/test 后 cp -r /tmp/test /usr/ 复制目录，目录必须加 -r。

6. mv：mv /usr/bashrc1 /usr/test/ 移入目录（保留原文件名）；mv /usr/test /usr/test2 在目标不存在时即为重命名。

7. rm：rm /usr/test2/bashrc1 删除文件；rm -r /usr/test2 递归删除目录，直接对目录用 rm 会报错。

8. cat：cat ~/.bashrc 一次性输出全部内容，适合小文件。

9. tac：tac ~/.bashrc 按行逆序输出，文件最后一行最先显示。

10. more：more ~/.bashrc 分页显示，空格翻页、回车下移一行、q 退出。

11. head：head -20 ~/.bashrc 显示前 20 行；head -n -50 ~/.bashrc 表示去掉末尾 50 行、输出其余全部。

12. tail：tail -20 ~/.bashrc 显示最后 20 行；tail -n +51 ~/.bashrc 从第 51 行输出到末尾。

13. touch：touch /tmp/hello 创建空文件，ls -l 可见访问、修改、状态时间均为当前时刻；touch -d "5 days ago" /tmp/hello 把时间改为 5 天前。

14. chown：sudo chown root /tmp/hello 把所有者改为 root，ls -l 第 3 列由 hadoop 变为 root。

15. find：find ~ -name ".bashrc" 在家目录按文件名查找，得到 /home/hadoop/.bashrc。

16. tar：sudo mkdir /test 后 sudo tar -zcvf /test.tar.gz /test（-z 用 gzip、-c 打包、-v 显示过程、-f 指定文件名）；sudo tar -zxvf /test.tar.gz -C /tmp 用 -C 解压到指定目录。

17. grep：grep -n "examples" ~/.bashrc 在文件中查找字符串并显示行号。

体会：rmdir 与 rm -r 的分工、head 的负号与 tail 的 +n 语义差异、cp -r 对目录的必要性，都是亲手敲一遍才会记住的细节。

二、启动 Hadoop 与 HDFS 基础操作

以 hadoop 用户登录后，依次执行 start-dfs.sh、start-yarn.sh，jps 可看到

NameNode、DataNode、SecondaryNameNode、ResourceManager、NodeManager 五个守护进程；Web UI http://localhost:9870 显示 HDFS 处于可用状态（安全模式自动退出）。随后完成：

（1）hdfs dfs -mkdir -p /user/hadoop 为 hadoop 用户创建 HDFS 用户目录，hdfs dfs -ls /user 可见 drwxr-xr-x   - hadoop supergroup 0 /user/hadoop。

（2）hdfs dfs -mkdir /user/hadoop/test 创建 test 文件夹，hdfs dfs -ls /user/hadoop 列出。

（3）hdfs dfs -put ~/.bashrc /user/hadoop/test 把本地文件上传到 test，hdfs dfs -ls /user/hadoop/test 显示 -rw-r--r-- 1 hadoop supergroup 3771 ... .bashrc。

（4）sudo hdfs dfs -get /user/hadoop/test /usr/local/hadoop/ 把 HDFS 上的 test 目录复制回本地 /usr/local/hadoop，ls -l /usr/local/hadoop/test 可见 .bashrc 已存在。

三、HDFS 文件操作的 Java 实现与 Shell 等价命令

程序 HDFSApp 以子命令形式实现 10 项功能，统一通过 FileSystem.get(URI.create("hdfs://localhost:9000"), conf, "hadoop") 获取文件系统对象。各项功能的设计要点如下：

1. 上传并处理已存在：覆盖用 fs.copyFromLocalFile(false, true, src, dst)（第 2 个参数 overwrite=true）；追加则用 fs.append(dst) 取得输出流后把本地文件内容拷入。Shell 对应 hdfs dfs -put -f 与 hdfs dfs -appendToFile，由用户选择。

2. 下载并自动重命名：copyToLocalFile 之前先判断本地目标目录下是否存在同名文件，若存在则按 name_1.ext、name_2.ext 递增挑选新名字。Shell 侧用 [-e 文件] 判断后 -get 到新文件名。

3. 输出文件内容到终端：fs.open(path) 得到 FSDataInputStream，用 BufferedReader 逐行读取并打印，最后关闭流；Shell 对应 hdfs dfs -cat。

4. 显示单个文件的权限、大小、创建时间、路径：fs.getFileStatus(path) 返回 FileStatus，分别取 getPermission()、getLen()、getReplication()、getModificationTime()、getPath()；Shell 对应 hdfs dfs -ls 或 hdfs dfs -stat "%A %b %y %n"。

5. 递归输出目录下所有文件信息：对 listStatus(dir) 的结果判断，目录则递归下降，文件则按上一项格式打印；Shell 对应 hdfs dfs -ls -R。

6. 创建/删除文件、父目录自动创建：先取 `file.getParent()` 判断，不存在则 `fs.mkdirs(parent)`，再 `fs.create(file)` 建文件；删除用 `fs.delete(file, false)`。Shell 对应 `hdfs dfs -mkdir -p` 加 `-touchz`，以及 `hdfs dfs -rm`。

7. 创建/删除目录：创建用 `fs.mkdirs(dir)`（自动补齐缺失父目录）；删除前用 `fs.listStatus(dir).length > 0` 判断是否为空，非空则拒绝删除并给出提示，空目录才 `fs.delete(dir, false)`。Shell 对应 `-mkdir -p` 与 `-rmdir`（`rmdir` 本身就只删空目录）。

8. 在开头或结尾追加内容：末尾用 `fs.append(path)` 写流即可；开头追加必须先用 `fs.open` 读出全文，再 `fs.create(path, true)` 重写（新内容在前、原内容在后），属于整文件重写。Shell 侧末尾用 `-appendToFile`，开头则 `-get` 下载、`cat` 拼接、`-put -f` 覆盖上传。

9. 删除指定文件：`fs.delete(path, false)`，删除前校验路径存在且不是目录；Shell：`hdfs dfs -rm`。

10. 移动文件：`fs.rename(src, dst)`，目标为目录时移入目录、目标为不存在的路径时即为改名；Shell：`hdfs dfs -mv`。

运行示例（查看属性、在文件开头追加内容）：

```
$ hadoop jar target/bigdata-lab1.jar cn.sdu.bigdata.lab1.hdfs.HDFSApp \
      stat /user/hadoop/lab1/hdfsdemo/.bashrc
权限       大小    副本   修改时间              路径
-rw-r--r-- 3771  1   2026-09-21 10:24:33  /user/hadoop/lab1/hdfsdemo/.bashrc
$ hadoop jar target/bigdata-lab1.jar cn.sdu.bigdata.lab1.hdfs.HDFSApp \
      append /user/hadoop/lab1/hdfsdemo/.bashrc "hello hdfs" head
已在文件开头追加内容（读取全文后重写）
hello hdfs
# 其后原   .bashrc 的内容
```

四、MapReduce 初级编程

4.1 文件的合并与去重。输入为样例文件 A、B，先上传到同一 HDFS 目录 `/user/hadoop/lab1/input/dedup` 作为作业输入。Mapper 以整行作为 key，并在输出前做空白归一化；Reducer 对每个 key 只输出一次，“合并 + 去重”一步完成：

```
// Mapper
String line = value.toString().trim();
if (line.isEmpty()) return;
outKey.set(line.replaceAll("\\s+", " "));      // 归一化连续空白
context.write(outKey, NullWritable.get());
// Reducer：相同 key 只输出一次
context.write(key, NullWritable.get());
```

作业命令与输出文件 C（共 9 行）：

```
hadoop jar $JAR cn.sdu.bigdata.lab1.mr.MergeDedup \
    /user/hadoop/lab1/input/dedup /user/hadoop/lab1/output/dedup
20170101 x
20170101 y
20170102 y
```

```
20170103 x
20170104 y
20170104 z
20170105 y
20170105 z
20170106 x
```

结果与指导书给出的样例完全一致：A、B 两文件共 11 行，合并去重后剩 9 行，重复的 `20170101 x`、`20170102 y`、`20170103 x`、`20170105 y` 各只保留一条。

4.2 对输入文件排序。把三个输入文件中的整数作为 key，利用 shuffle 排序；Reducer 用一个成员变量 `rank` 累加名次，并强制 `setNumReduceTasks(1)` 保证全局有序：

```
// Reducer
rank++;
context.write(new IntWritable(rank), new IntWritable(key.get()));
// Driver
job.setNumReduceTasks(1);        // 全局有序要求只有一个 Reducer
```

输入为 33、37、12、40，4、16、39、5，1、45、25 共 11 个整数，输出为“名次 整数”：

```
1   1       5   16      9   39
2   4       6   25      10  40
3   5       7   33      11  45
4   12      8   37
```

（实际输出每行一组，此处按三列并排以便节省篇幅。）名次从 1 连续到 11，数值严格升序，与样例一致。

4.3 child-parent 表的信息挖掘。把该表视为关系 R(child, parent)，祖孙关系就是 R 与自身的连接，连接条件为 R1.parent = R2.child。Mapper 对每条记录发出两条带标记的记录，Reducer 对同一 key 下的两类列表求笛卡尔积：

```
// Mapper：同一结点既可能作为父辈，也可能作为子辈
context.write(new Text(parent), new Text("1:" + child));   // 孙辈候选
context.write(new Text(child),  new Text("2:" + parent));  // 祖辈候选
// Reducer：笛卡尔
for (String gc : grandChildren)
    for (String gp : grandParents)
        context.write(new Text(gc), new Text(gp));
```

以 key = Lucy 为例，标记 1 的记录给出孙辈候选 Steven、Jone，标记 2 的记录给出祖辈候选 Mary、Frank，笛卡尔积得到 (Steven, Mary)、(Steven, Frank)、(Jone, Mary)、(Jone, Frank) 四条。全表共输出 12 条祖孙关系（按 key 字典序排列，共 12 行）：

```
Mark    Alice       Jone    Alice       Jone    Frank
Mark    Jesse       Jone    Jesse       Jone    Mary
Philip  Alice       Steven  Alice       Steven  Frank
Philip  Jesse       Steven  Jesse       Steven  Mary
```

与指导书样例对照，12 组祖孙关系完全相同，只是行的排列顺序不同——这是因为样例按“孙辈分组”排列，而作业输出按 reduce 的 key 字典序排列，集合意义上的结果是等价的。

以上三个 MapReduce 程序与第 1、2 部分的全部脚本已整理为 Maven 工程，上传至 GitHub 公开仓库：https://github.com/<GitHub 用户名>/bigdata-lab1（Public）。目录约定：scripts/ 放 Shell 脚本，src/main/java 放 Java 源码，input/ 为样例输入，out/ 为输出结果，tools/local_check.py 为不依赖 Hadoop 的本地逻辑校验脚本；编译命令 mvn -q clean package 生成 target/bigdata-lab1.jar。

结论分析与体会：

（1）Linux 部分。Shell 命令的短选项可以合并（tar -zcvf），同一种“删除”被拆成 rmdir（空目录）、rm（文件）、rm -r（目录树）三种语义，head -n -50 与 tail -n +51 分别用负号和加号表示“排除末尾”与“从第 n 行起”。这些约定只有实际操作过一遍才会形成直觉，单纯背命令很容易在参数符号上出错。

（2）HDFS 部分。写 Java 程序之后才真正看清 hdfs dfs 只是 API 的外壳，两者一一对应；也真正理解了“一次写入、多次读取”的含义——追加只能加在末尾，若要在开头插内容，只能读出全文重写，代价与文件大小成正比。这正是 HDFS 不适合频繁修改小文件的原因，也解释了为什么大数据场景更倾向于“写一次、永不改”的数据组织方式。

（3）MapReduce 部分。三个问题表面上分别是去重、排序和连接，但最终都归结为“如何设计 key”：key 选对了，排序、分组、连接这些框架能力就自动为你服务。如果没意识到 shuffle 会按 key 分组，很容易把去重写成“循环比较两两判断”，把排序写成单机内存排序，既低效又无法扩展到大数据。

（4）遇到的问题与收获。最初编写排序程序时没有设置 setNumReduceTasks(1)，在默认 1 个 reducer 时结果正常，改成 2 个后输出分成两段各自有序的数据、名次重复从 1 开始，才意识到“分区内有序”不等于“全局有序”。另外首次运行去重程序时 20170101 x 输出了两条，排查发现是 A、B 两个样例文件字段间空格个数不同导致 key 不一致，加入空白归一化后解决。总的收获是：从 Linux 命令到 HDFS API 再到 MapReduce 编程模型，对 Hadoop 的分层结构有了整体认识——底层是 Linux 文件系统，中间是 HDFS 的分布式存储，上层是 MapReduce 计算框架；把代码整理为 Maven 工程、把操作写成可重复执行的 Shell 脚本，比零散敲命令更可靠，也便于复查与提交。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

问题一：为什么不能在 HDFS 文件的“开头”追加内容？应如何实现？

答：HDFS 只允许在文件末尾顺序追加（且需开启 `dfs.support.append`），写入时客户端定位到最后一个数据块的末尾继续写。文件由若干固定大小的块组成，块在 DataNode 上不可原地修改，任何“中间插入”都会改变其后所有块的偏移量，因此框架不提供随机写接口。实现“开头追加”只能让程序先 `fs.open` 读出全部内容，再 `fs.create(path, true)` 新建文件并按“新内容 + 原内容”的顺序重写，本质是整文件替换，复杂度为 O(n)；小文件可以接受，大文件应改为“另建新文件 + 调整读取逻辑”。Shell 下的等价做法是 `hdfs dfs -get` 下载、`cat` 拼接、`hdfs dfs -put -f` 覆盖上传。

问题二：全局排序为什么必须用单个 Reducer？数据量大时怎么办？

答：shuffle 只保证“分区内有序”，即每个 reduce 拿到的 key 是有序的，但不同 reduce 之间没有任何顺序约定，而名次必须全局连续，所以只能让一个 reducer 看到全部 key；代价是无法并行、易成为瓶颈。数据量大时有两种改进：一是使用 `TotalOrderPartitioner` 配合 `InputSampler` 预先采样并划分互不重叠的 key 区间，各 reduce 负责一个区间，输出天然全局有序；二是先由多个 reduce 做局部排序，再用一个只做归并的小作业汇总。本实验只有 11 个整数，单 reducer 最简单可靠。

问题三：上传或创建文件时出现 “Name node is in safe mode” 或 “Permission denied”，如何排查？

答：前者表示 NameNode 启动后仍处于安全模式（只读状态），正在等待 DataNode 汇报块信息。先用 `jps` 确认 DataNode 是否启动、用 `hdfs dfsadmin -report` 查看存活节点与块比例；若只是刚启动、块比例未达阈值，等待其自动退出；若确认不存在缺块，可执行 `hdfs dfsadmin -safemode leave` 手动退出。后者通常是权限或属主问题：用 `hdfs dfs -ls` 查看目标目录的属主，hadoop 用户只对自己家目录 `/user/hadoop` 有写权限，可先 `hdfs dfs -mkdir -p /user/hadoop`，或用 `hdfs dfs -chown -R hadoop:supergroup /user/hadoop` 修改属主；此外 `fs.defaultFS` 配置错误、`dfs.namenode.name.dir` 未格式化也会引起类似报错。